Arrays versus Listas Ligadas (Linked Lists)

Prof. Pablo De Chiaro

algoritmos e estrutura de dados 2 (AEDII)

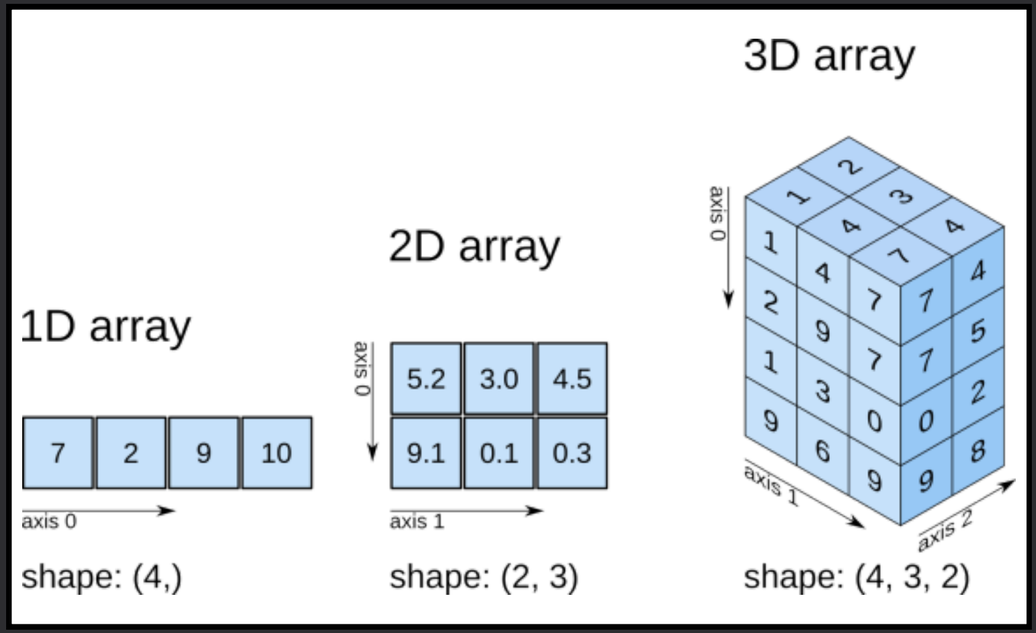
_aula 6

Objetivos da Aula

- compreender o que é uma lista ligada e como ela difere de um array
- explicar por que arrays favorecem acesso por índice e listas ligadas favorecem certas inserções/remoções
- implementar operações básicas em listas ligadas
- comparar a complexidade de operações nas duas estruturas
- identificar em que cenário uma estrutura tende a ser mais adequada que a outra

Retomando

- **em arrays**, o acesso por índice tende a ser O(1)
- busca por valor tende a ser O(n)
- inserções e remoções no início/meio podem exigir deslocamento dos elementos, levando a O(n)



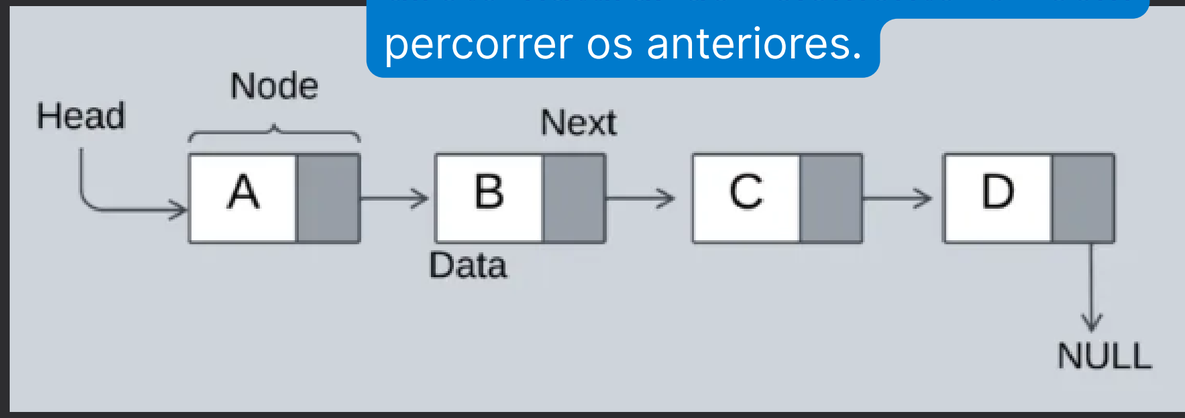
“Se inserir no início do array é custoso porque precisamos mover dados, existe uma estrutura em que isso seja mais barato?”

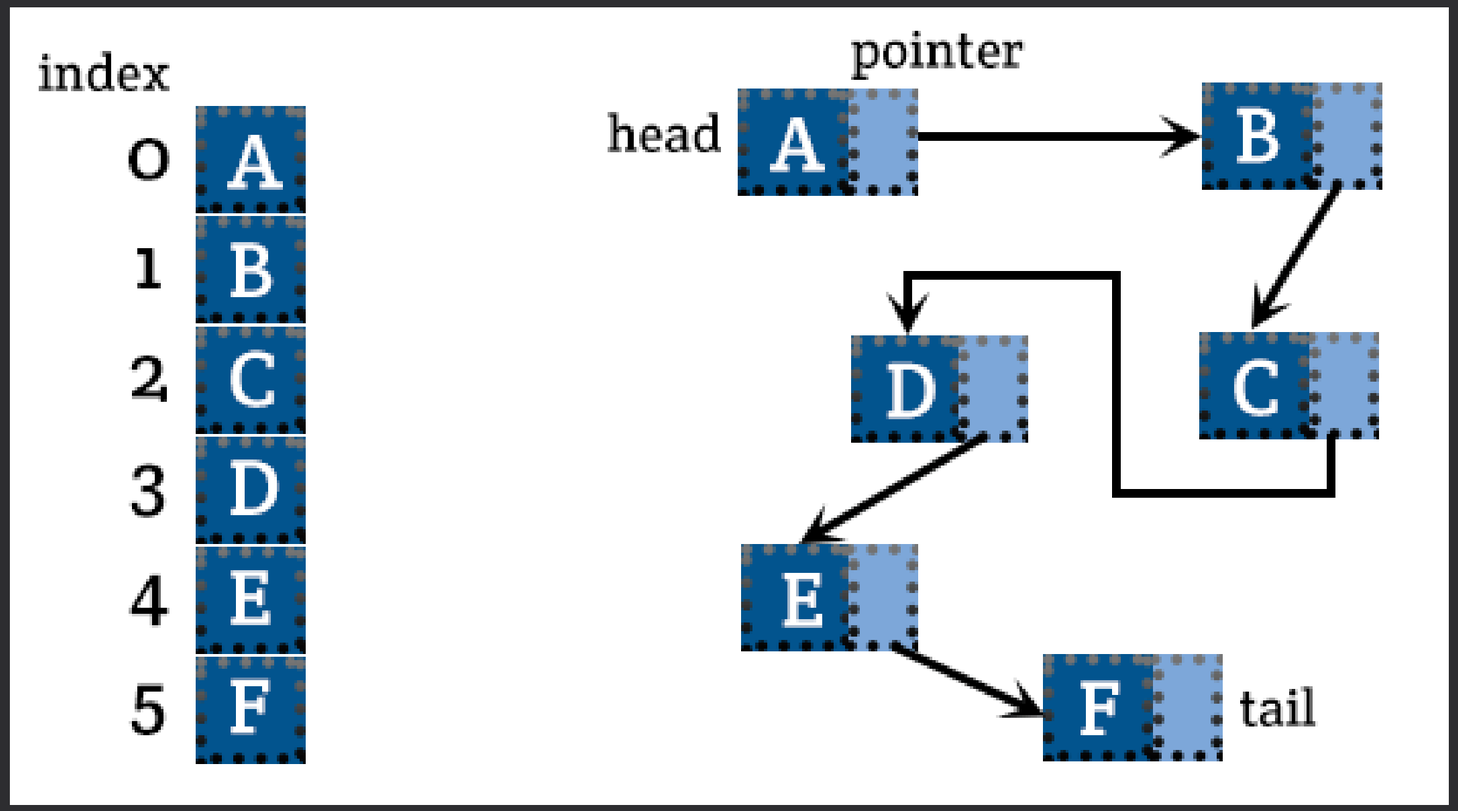
Introdução conceitual a Linked Lists

- no array, os elementos estão em posições indexadas e, no modelo conceitual ensinado, armazenados de forma contígua

- na **linked list**, cada elemento é um nó
- cada nó guarda:
  - um valor
  - uma referência para o próximo nó

na lista ligada, não existe acesso direto natural ao “elemento C” sem percorrer os anteriores.




index
0
1
2
3
4
5
A
B
C
D
E
F
pointer
head
A
B
D
C
E
F
tail

_metáfora

array = gavetas numeradas

linked list = vagões conectados por engates

Operação	Array	Linked List
Acesso por índice	O(1)	O(n)
Busca por valor	O(n)	O(n)
Inserção no início	O(n)	O(1)
Remoção no início	O(n)	O(1)
Inserção no fim	O(1)*	O(n) / O(1)**
Remoção no fim	O(1)*	O(n)
Inserção no meio	O(n)	O(n) para localizar + O(1) para ligar
Remoção no meio	O(n)	O(n) para localizar + O(1) para religar

* Assumindo que há espaço disponível e capacidade não precisa ser realocada.
** O(n) sem ponteiro de cauda, O(1) com ponteiro de cauda.

RESUMO

```python
class No:
    def __init__(self, valor):
        self.valor = valor
        self.proximo = None


class ListaLigada:
    def __init__(self):
        self.head = None

    def inserir_no_inicio(self, valor):
        novo = No(valor)
        novo.proximo = self.head
        self.head = novo

    def percorrer(self):
        atual = self.head
        while atual is not None:
            print(atual.valor, end=" -> ")
            atual = atual.proximo
        print("None")

```

https://colab.research.google.com/drive/1rEgBlmSoR7aonGnEVF8odbOegfm-Ct0H?usp=sharing